

AstraMarket Sunum Rehberi

Ahsen, Furkan, Egehan — 15-20 dakika hocaya sunum

TIKLA = tıklanacak şey | **SÖYLE** = birebir cümle | **DOSYA** = açılacak dosya | **SATIR** = satır numarası |
KOD = vurgulanacak kod

Sunumdan 15 Dakika Önce Hazırlık

1. Servisleri başlat

```
cd ~/Desktop/web_prog  
./start.sh
```

Yeşil: MySQL, Backend, Frontend çıkmalı.

2. Veritabanını temizle (sıfır verisi için)

```
cd backend  
npm run init-db && node db/add-fake-users.js
```

3. VS Code'da açık olması gereken dosyalar (alttaki tablardan sıralı)

- frontend/src/components/Navbar.jsx
- frontend/src/pages/Home.jsx
- frontend/src/components/LogoBackground.jsx
- frontend/src/components/ProtectedRoute.jsx
- backend/middleware/auth.js
- backend/routes/auth.js
- backend/routes/siparişler.js
- backend/db/schema.sql
- backend/routes/raporlar.js
- queries.sql (proje kökü)

4. Tarayıcı sekme sırası

- http://localhost:5173 (site)
- http://localhost/phpmyadmin veya DBeaver
- VS Code (alt+tab ile geçiş)

5. sessionStorage temizle (intro tekrar gözüksün)

F12 console'da: `sessionStorage.clear()` → sayfayı kapa.

Rol Dağılımı

Kişi	Bölüm	Süre
Ahsen	Açılış + Müşteri akışı + Checkout	~6 dk

Furkan	Admin Paneli + CRUD + Frontend kodu	~6 dk
Egehan	Veritabanı + 4 SQL sorgusu + Transaction	~7 dk

AHSEN — Açılış + Müşteri Akışı (6 dk)

0:00-1:00 — Açılış Konuşması

SÖYLE: "Merhaba hocam, ben Ahsen. Grubumuzla birlikte AstraMarket adında, Hepsiburada benzeri bir e-ticaret otomasyonu geliştirdik. Projemizde bir müşteri olarak alışveriş yapabilir, ürünlere yorum bırakabilir, sipariş verebilirsiniz. Ayrıca admin panelinden tüm ürünleri, kullanıcıları ve kategorileri yönetebilirsiniz."

SÖYLE: "Teknoloji stackimizi kısaca özetleyeyim: Frontend tarafında React ve Vite kullandık, tasarım için React-Bootstrap tercih ettik. Backend tarafında Node.js ve Express frameworku ile REST API yazdık. Veritabanı olarak MySQL, yani MariaDB kullanıyoruz. Kullanıcı kimlik doğrulaması için JWT, yani JSON Web Token kullanıyoruz ve şifreleri bcryptjs kütüphanesiyle hash'leyerek veritabanında saklıyoruz. Yani şifre hiçbir zaman düz metin olarak tutulmuyor."

SÖYLE: "Projemizde toplam 6 veritabanı tablomuz, 20 farklı sayfamız, 18'den fazla Bootstrap componentimiz var. Tüm tablolarda CRUD işlemleri çalışıyor ve 4 tane gelişmiş SQL sorgumuz mevcut. Bunlara ek olarak Premium üyelik sistemi, öğrenci belge doğrulama akışı ve kozmik görsel efektler de ekledik. Şimdi siteden başlayalım."

1:00-2:00 — Intro + Görsel Efektler

TIKLA: Tarayıcı sekmesi 1 (localhost:5173) Intro açıksa bekle, harf-harf gelen tagline'i işaret et.

SÖYLE: "Site ilk açıldığında sinematik bir intro ekranıyla karşılaşıyorsunuz. Burada AstraMarket logosu ve harf harf beliren bir tagline var. Kullanıcıya 3 seçenek sunuluyor: Giriş Yap, Kayıt Ol veya Misafir olarak devam et. Misafir seçeneğiyle kayıt olmadan da siteyi gezebilirsiniz, ama yorum yazmak ve sipariş vermek için giriş gerekiyor."

TIKLA: "Misafir olarak devam et" tıkla → Anasayfa açılır.

SÖYLE: "Anasayfada gördüğünüz gibi arka planda kozmik görsel efektler çalışıyor. Perspektif grid tüneli, uçan yıldızlar ve scroll'a tepki veren orbital bir sistem var. Sayfayı aşağı kaydıldıkça warp streaks, yani ışık izleri efekti devreye giriyor. Bunların hepsi tamamen CSS transform ve JavaScript ile yapıldı, Three.js gibi ağır bir kütüphane kullanmadık. GPU hızlandırmalı oldukları için performansı düşürmüyor."

TIKLA: Sayfayı yavaşça aşağı kaydır — warp streaks efektini göster, mouse'u hareket ettir.

2:00-2:45 — Kategori Dropdown + Filtreleme

TIKLA: Navbar'da "Kategoriler" tıkla.

SÖYLE: "Navbar'da Kategoriler dropdown'u var. 6 kategorimiz mevcut: Elektronik, Giyim, Ev ve Yaşam, Spor, Kitap ve Kozmetik. Burada ilginç bir teknik detay var — React-Bootstrap'in kendi Dropdown componentini denedik ama CSS spesifiklik sorunları çıkardı, dropdown menünün konumu ve stili bozuluyordu. Bu yüzden sıfırdan kendi custom dropdown componentimizi yazdık."

TIKLA: "Elektronik" seç.

SÖYLE: "Elektronik kategorisine tıkladığımda URL'e kategori parametresi eklendi, gördüğünüz gibi soru işareti kategori eşittir 1 yazıyor. Sayfa bu parametreyi okuyor, otomatik olarak filtre uyguluyor ve ürünler bölümüne smooth scroll ile iniyor. Aynı mantık tüm kategoriler için çalışıyor."

2:45-3:45 — Ürün Detay + Yorum

TIKLA: Bir ürüne tıkla (örn. iPhone).

SÖYLE: "Ürün detay sayfasına geldik. Burada ürünün görseli, adı, fiyatı, yıldız puanı, açıklaması ve stok durumu görünüyor. Aşağıda adet seçici var, artı ve eksi butonlarıyla miktar ayarlayıp sepete ekleyebilirsiniz. Stoktan fazla seçemezsiniz, kontrol var."

SÖYLE: "Sayfanın alt kısmında yorumlar bölümü var. Şimdi yorum eklemek için giriş yapmam gerekiyor."

TIKLA: Sağ üstte "Giriş / Kayıt" tıkla.

SÖYLE: "Giriş sayfamızda email ve şifre alanları var. Demo için hazır bir müşteri hesabıyla giriş yapıyorum."

TIKLA: ayse@mail.com / müşteri123 yaz → Giriş Yap.

SÖYLE: "Giriş başarılı oldu. Arka planda ne oldu diye anlatayım: backend'e email ve şifre gönderildi, backend şifrenin bcrypt hash'ini veritabanındakiyle karşılaştırdı, eşleştirdi ve bize bir JWT token döndürdü. Bu token localStorage'a kaydedildi ve bundan sonra her API isteğinde bu token header'da gönderiliyor."

TIKLA: Tekrar aynı ürüne git → yorum kutusuna "Harika ürün, çok memnunum" yaz → 5 yıldız seç → Gönder.

SÖYLE: "Yorum eklendi ve hemen listeye düştü. Dikkat ederseniz kendi yorumumun yanında 'Sizin yorumunuz' etiketi ve kırmızı bir silme butonu görünüyor. Sistem kullanıcı ID'sini kontrol ediyor — sadece kendi yorumunuzu silebilirsiniz. Admin ise tüm yorumları silebilir, onu Furkan gösterecek."

3:45-5:15 — Sepet → Checkout → Sipariş

TIKLA: Ürüne artı butonuyla adet 2 yap → Sepete Ekle.

SÖYLE: "Sepete 2 adet ekledim. Navbar'daki sepet ikonundaki badge bounce animasyonuyla 2'ye güncellendi. Bu badge React-Bootstrap'in Badge componenti, animasyon ise CSS keyframe ile."

TIKLA: Sepet ikonuna tıkla.

SÖYLE: "Sepet sayfasındayız. Burada ürün görseli, adı, birim fiyatı, adet kontrolü ve ara toplam görünüyor. Altta kargo ücreti ve genel toplam hesaplanıyor. Dikkat ederseniz 500 TL üzeri ücretsiz kargo uyarısı var — toplam 500 TL'yi geçerse kargo otomatik sıfırlanacak."

TIKLA: "Ödemeye Geç" butonu tıkla.

SÖYLE: "Çok adımlı checkout sayfamıza geçtik. Üstteki stepper'da 4 adım görünüyor: Sepet Özeti, Teslimat Bilgileri, Ödeme ve Onay. Şimdi teslimat bilgilerini dolduralım."

Adım 1: Teslimat

Formu doldur:

- Ad: Ayse, Soyad: Yılmaz, Telefon: 0532 111 22 33
- Adres: Bağdat Caddesi No: 42
- Şehir: İstanbul, İlçe: Kadıköy, Posta: 34710

SÖYLE: "Teslimat formunu doldurdum. Tüm alanlar validasyonlu — boş bırakınca kırmızı uyarı çıkıyor."

TIKLA: "Ödemeye Geç" tıkla.

Adım 2: Ödeme

SÖYLE: "Ödeme adımında 3 farklı ödeme yöntemi sunuyoruz: Kredi Kartı, Banka Havalesi ve Kapıda Ödeme. Kredi kartı seçili olduğu için kart formu görünüyor. Tabii bu bir demo projesi, gerçek ödeme işlenmiyor — sarı uyarı kutusunda da bunu belirtiyoruz."

Kart bilgileri (sahte):

- İsim: AYSE YILMAZ
- No: 4444 4444 4444 4444
- SKT: 12/28
- CVV: 123

TIKLA: "Siparişi Onayla" tıkla.

SÖYLE: "Şimdi arka planda çok önemli bir şey oldu. Backend'de bir MySQL transaction çalıştı. Öncelikle sipariş edilen ürünlerin satırları SELECT FOR UPDATE ile kilitlendi — başka bir sipariş aynı ürünlere erişmek isterse beklemek zorunda. Sonra stok kontrolü yapıldı, yeterliyse siparişler tablosuna bir kayıt, sipariş_detaylari tablosuna her ürün için bir kayıt eklendi ve ürünlerin stokları düşürüldü. Hepsi aynı transaction içinde — biri başarısız olursa tümünü geri alıyor. Transaction detayını Egehan kod üzerinden gösterecek."

Sipariş Tamamlandı sayfası

SÖYLE: "Sipariş tamamlandı sayfasına yönlendirildik. Konfeti animasyonu, büyük yeşil bir onay işareti, sipariş numarası, toplam tutar ve tahmini teslimat tarihi görünüyor. Konfeti efekti CSS keyframe ile yapıldı."

TIKLA: "Siparişlerimi Görüntüle" tıkla.

SÖYLE: "Siparişlerim sayfasında az önce verdiğimiz sipariş en üstte görünüyor. Durumu 'Hazırlanıyor' yazıyor. Accordion'a tıklayınca siparişin detaylarını, yani hangi üründen kaç adet alındığı ve birim fiyatları görebilirsiniz."

5:15-6:00 — Diğer Sayfalar (kısa tur)

TIKLA: Navbar'da "Super Fiyat, Super Teklif" tıkla.

SÖYLE: "İndirimli ürünler sayfamız. Ürünler 3 kategoride gruplanmış: yüzde 40 ve üzeri, yüzde 30 ila 39, yüzde 20 ila 29. Üstte bir geri sayım sayacı var, kampanyanın ne zaman bittiğini gösteriyor."

TIKLA: "Kampanyalar" tıkla.

SÖYLE: "6 farklı kampanya kartı görünüyor. Her birinin kendine özel rengi ve açıklaması var."

TIKLA: "Müşteri Hizmetleri" → "AstraMarket Premium" tıkla.

SÖYLE: "Bu bizim Premium üyelik sistemimiz. 3 farklı plan var: Standart ayda 39 lira 90 kuruş, Öğrenci planı yüzde 50 indirimli ayda 19 lira 90 kuruş, ve Aile planı ayda 59 lira 90 kuruş. Öğrenci planında ekstra bir doğrulama adımı var — üniversite adı, bölüm, öğrenci numarası girmeniz ve öğrenci belgenizi PDF veya resim olarak yüklemeniz gerekiyor. 24 saat içinde doğrulama yapılacağı belirtiliyor."

TIKLA: Geri dön → "Sıkça Sorulan Sorular" tıkla.

SÖYLE: "SSS sayfamız. 5 farklı kategori altında sorular accordion yapısıyla listeleniyor. React-Bootstrap'in Accordion componenti kullanıldı. Şimdi Furkan admin panelini gösterecek, buyurun Furkan."

FURKAN — Admin Paneli + CRUD (6 dk)

6:00-7:00 — Admin Girişi + Route Koruması

TIKLA: Sağ üstte "Ayse" dropdown → "Çıkış Yap" tıkla.

SÖYLE: "Ben Furkan. Şimdi admin tarafını göstereceğim. Önce çıkış yaptım, localStorage'daki JWT token silindi ve giriş sayfasına yönlendirildim."

TIKLA: "Giriş / Kayıt" → admin@eticaret.com / admin123 yaz → Giriş Yap.

SÖYLE: "Admin hesabıyla giriş yaptım. Dikkat ederseniz sistem beni otomatik olarak admin paneline yönlendirdi. Bunun sebebi login sonrası kullanıcının rolünü kontrol ediyoruz — rolü admin ise admin paneline, müşteri ise anasayfaya yönlendiriyoruz."

SÖYLE: "Burada güvenlik açısından çok önemli bir nokta var. Normal bir müşteri hesabıyla admin panelinin URL'ine gitmeye çalışsanız ne olur? Sistem sizi giriş sayfasına atıyor ve 'Bu sayfaya erişim yetkiniz yok' uyarısı gösteriyor. Bu koruma hem frontend'de hem backend'de var."

KOD GÖSTERİMİ (Kritik!)

DOSYA: VS Code ac → frontend/src/components/ProtectedRoute.jsx Tüm dosyayı göster:

```
export default function ProtectedRoute({ children, adminOnly = false }) {
  const { user } = useAuth();
  const loc = useLocation();
  if (!user) return <Navigate to="/login" state={{ from: loc }} replace />;
  if (adminOnly && user.rol !== 'admin')
    return <Navigate to="/login" state={{ yetki: 'Bu sayfaya erişim yetkiniz yok' }}
  replace />;
  return children;
}
```

SÖYLE: "Bu ProtectedRoute componenti iki katmanlı kontrol yapıyor. Birincisi: kullanıcı giriş yapmamışsa login sayfasına yönlendiriyor. İkincisi: adminOnly parametresi true ise ve kullanıcının rolü admin değilse, yine login'e atıp uyarı mesajı gösteriyor. React Router'in Navigate componentiyile çalışıyor."

DOSYA: backend/middleware/auth.js ac SATIR 16-21 göster:

```
function adminRequired(req, res, next) {
  authRequired(req, res, () => {
```

```
    if (req.user.rol !== 'admin') return res.status(403).json({ hata: 'Admin yetkisi gerekli' });  
    next();  
  });  
}
```

SÖYLE: "Sadece frontend kontrolüne güvenmiyoruz çünkü frontend atlatılabilir. Backend tarafında da aynı kontrolü yapıyoruz. Bu adminRequired middleware'i önce JWT token'i doğruluyor, sonra kullanıcının rolünü kontrol ediyor. Admin değilse 403 Forbidden dönüyor. Tüm admin API endpoint'leri bu middleware'den geçiyor."

7:00-8:30 — Ürün CRUD (Create + Update + Delete)

TIKLA: Sol sidebar'da "Ürün Yönetimi" tıkla.

SÖYLE: "Ürün yönetimi sayfasındayız. Tabloda 100 ürün listeleniyor. Her üründe ad, fiyat, stok, kategori görünüyor ve sağ tarafta düzenle ve sil butonları var. Şimdi sırayla ekleme, güncelleme ve silme işlemlerini göstereceğim."

CREATE

TIKLA: "+ Yeni Ürün" butonuna tıkla → Modal açılır.

SÖYLE: "React-Bootstrap'in Modal componenti açıldı. Ürün adı, fiyat, stok miktarı, kategori seçimi ve açıklama alanları var."

Form doldur:

- Ad: Demo Ürün, Fiyat: 999, Stok: 10, Kategori: Elektronik, Açıklama: Sunum için demo

TIKLA: Kaydet tıkla.

SÖYLE: "Ürün başarıyla eklendi. Arka planda POST /api/ürünler endpoint'ine istek gitti."

KOD GÖSTERİMİ

DOSYA: VS Code → backend/routes/ürünler.js SATIR 36-43 göster:

```
router.post('/', adminRequired, async (req, res) => {  
  const { ad, fiyat, stok, kategori_id, resim_url, açıklama } = req.body;  
  const [r] = await pool.query(  
    'INSERT INTO ürünler (ad, fiyat, stok, kategori_id, resim_url, açıklama) VALUES  
(?, ?, ?, ?, ?, ?)',  
    [ad, fiyat, stok, kategori_id || null, resim_url || null, açıklama || null]  
  );  
  res.json({ id: r.insertId });  
});
```

SÖYLE: "Dikkat ederseniz route'un başında adminRequired middleware'i var — sadece admin bu endpoint'e erişebilir. SQL sorgusunda soru işareti ile parametre kullanıyoruz, buna prepared statement deniyor. Bu SQL injection saldırılarına karşı koruma sağlıyor. Kullanıcıdan gelen veri doğrudan SQL'e yazılmıyor, veritabanı motoru parametreleri güvenli şekilde yerleştiriyor."

UPDATE

TIKLA: Listede yeni oluşan "Demo Ürün"ün Düzenle butonuna tıkla → Fiyatı 799 yap → Kaydet.

SÖYLE: "Fiyatı 999'dan 799'a güncelledim. PUT /api/ürünler/:id endpoint'i çağırıldı. Aynı prepared statement yaklaşımı burada da kullanılıyor, sadece INSERT yerine UPDATE sorgusu var."

DELETE

TIKLA: Aynı ürünün Sil butonuna tıkla → Onay modali açılır → "Evet, Sil" tıkla.

SÖYLE: "Silme işleminden önce onay modali çıkıyor — kullanıcının yanlışlıkla silmesini engelliyoruz. Onayladıktan sonra DELETE /api/ürünler/:id endpoint'i çağırıldı ve ürün veritabanından silindi. Böylece ürünler tablosundaki tam CRUD döngüsünü — yani Create, Read, Update, Delete — göstermiş olduk."

8:30-10:00 — Kullanıcı & Kategori CRUD + Yorum Silme

TIKLA: Sol sidebar'da "Kullanıcı & Kategori" tıkla → Kullanıcılar sekmesini göster.

SÖYLE: "Kullanıcı yönetimi sayfasında 20 kullanıcı listeleniyor — 1 admin ve 19 müşteri. Tabloda ad, email, rol ve kayıt tarihi görünüyor. Kullanıcı düzenleyebilir veya silebilirsiniz."

TIKLA: Kategoriler sekmesine geç.

SÖYLE: "Kategoriler sekmesinde 6 kategorimiz var. Şimdi hızlıca bir kategori ekleyip sileceğim."

TIKLA: "+ Yeni Kategori" tıkla → "Test Kategorisi" yaz → Kaydet.

SÖYLE: "Kategori eklendi, tablonun sonunda görünüyor."

TIKLA: Az önce eklenen "Test Kategorisi"nin Sil butonuna tıkla → Onayla.

SÖYLE: "Silindi. Burada önemli bir nokta: eğer bu kategoriye bağlı ürünler olsaydı, silme işleminde veritabanındaki ON DELETE SET NULL kurali devreye girecekti — ürünler silinmeyecek, sadece kategori_id alanları NULL olacaktı. Bunu Egehan schema kısmında detaylı açıklayacak."

SÖYLE: "Böylece kullanıcılar, kategoriler ve ürünler tablolarında CRUD işlemlerini gösterdik. Yorumlar tablosunda da CRUD var."

Yorum CRUD

TIKLA: Ust menu'den siteye dön → herhangi bir ürünün detay sayfasına git.

SÖYLE: "Admin olarak giriş yaptığım için tüm yorumların yanında kırmızı sil butonu görünüyor. Normal müşteri sadece kendi yorumunu görüyor."

KOD GÖSTERİMİ

DOSYA: VS Code → backend/routes/yorumlar.js SATIR 20-31 göster:

```
router.delete('/:id', authRequired, async (req, res) => {
  const [rows] = await pool.query('SELECT kullanıcı_id FROM yorumlar WHERE id = ?',
    [req.params.id]);
  if (!rows.length) return res.status(404).json({ hata: 'Yorum bulunamadi' });
  if (rows[0].kullanıcı_id !== req.user.id && req.user.rol !== 'admin') {
```

```
    return res.status(403).json({ hata: 'Yetkiniz yok' });
  }
  await pool.query('DELETE FROM yorumlar WHERE id = ?', [req.params.id]);
  res.json({ ok: true });
});
```

SÖYLE: "Yorum silme endpoint'inde önce yorumun kime ait olduğunu sorguluyoruz. Sonra iki koşul kontrol ediliyor: ya yorumun sahibi olmalısınız, ya da admin olmalısınız. İkisi de değilseniz 403 Forbidden dönüyor. Bu sayede bir müşteri başka birinin yorumunu silemiyor."

10:00-10:30 — Tasarım + Tema

TIKLA: Navbar'da tema toggle butonuna (gunes/ay ikonu) bas.

SÖYLE: "Sitemizde dark ve light mode desteği var. Bu butonla geçiş yapabilirsiniz. Tema tercihi localStorage'da saklanıyor, yani sayfayı kapatıp açsanız bile seçtiğiniz tema kalıcı oluyor. Eğer daha önce hic seçmediyseniz, işletim sisteminizin tema tercihinin otomatik algılayıp onu uyguluyor. Tüm 20 sayfa responsive tasarıma sahip, mobilde de düzgün çalışıyor. Şimdi en teknik bölüm için sözü Egehan'a bırakıyorum."

TIKLA: Tekrar dark moda dön.

EGEHAN — Veritabanı + SQL (7 dk) — EN TEKNİK

10:30-11:15 — DB Bağlantı + Tablo Yapısı

SÖYLE: "Ben Egehan. Şimdi projenin veritabanı tarafını, SQL sorgularını ve transaction mantığını göstereceğim."

TIKLA: phpMyAdmin sekmesine geç (sekme 2) → eticaret_db sec.

SÖYLE: "Veritabanımız MySQL, daha doğrusu MariaDB üzerinde çalışıyor. Toplam 6 tablomuz var. Kısaca listeleyelim: kullanıcılar tablosunda 20 kayıt var, 1 admin 19 müşteri. Kategoriler tablosunda 6 kayıt, ürünler tablosunda 100 ürün var. Siparişler tablosunda şimdiye kadar verilen siparişler, sipariş_detaylari tablosunda her siparişin içindeki ürünlerin detayı tutuluyor. Son olarak yorumlar tablosunda kullanıcıların bıraktığı 150 yorum var. Tüm tablolar InnoDB storage engine kullanıyor, bu da transaction ve foreign key desteği sağlıyor."

SÖYLE: "Tüm tablolar utf8mb4 character set ve utf8mb4_unicode_ci collation kullanıyor. Neden utf8mb4? Çünkü standart utf8 sadece 3 byte'lik karakterleri destekler ama utf8mb4, yani UTF-8 Mobile 4, emoji dahil tüm Unicode karakterlerini kapsar."

11:15-12:15 — Schema (CREATE TABLE'lar)

DOSYA: VS Code ac → backend/db/schema.sql

A) Database oluşturma

SATIR 1-4 göster:


```
DROP DATABASE IF EXISTS eticaret_db;
CREATE DATABASE eticaret_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE eticaret_db;
```

SÖYLE: "İlk 3 satırda veritabanını sıfırdan oluşturuyoruz. DROP IF EXISTS ile önceki veritabanı varsa siliniyor, sonra yenisi oluşturuluyor."

B) kullanıcılar tablosu

SATIR 7-14 göster:

```
CREATE TABLE kullanıcılar (
  id INT AUTO_INCREMENT PRIMARY KEY,
  ad VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  şifre VARCHAR(255) NOT NULL,
  rol ENUM('admin', 'müşteri') DEFAULT 'müşteri',
  oluşturma_tarihi TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

SÖYLE: "Kullanıcılar tablosunu satırsal inceleyelim. id alanı INT tipinde, AUTO_INCREMENT ile her yeni kayıta otomatik artıyor ve PRIMARY KEY olarak tanımlı. Email alanı UNIQUE constraint'e sahip — aynı email ile iki kayıt oluşturulamaz, veritabanı bunu garanti ediyor. Şifre alanı VARCHAR 255 çünkü bcryptjs hash'i 60 karakter uzunluğunda oluyor, 255 yeterli alan bırakıyor. Rol alanı ENUM tipinde — sadece 'admin' veya 'müşteri' değerlerinden birini alabilir, başka bir değer girilirse MySQL hata verir. DEFAULT olarak 'müşteri' atanıyor. ENGINE=InnoDB ile transaction desteği sağlıyoruz."

C) ürünler tablosu + Foreign Key

SATIR 23-34 göster:

```
CREATE TABLE ürünler (
  ...
  FOREIGN KEY (kategori_id) REFERENCES kategoriler(id) ON DELETE SET NULL
) ENGINE=InnoDB;
```

SÖYLE: "Ürünler tablosunda foreign key kullandık. kategori_id alanı kategoriler tablosunun id'sine referans veriyor. ON DELETE SET NULL kuralı diyor ki: eğer bir kategori silinirse, o kategoriye ait ürünler silinmesin — sadece kategori_id alanları NULL olsun. Bu veri tutarlılığı için çok önemli, yetim kayıtlar oluşmuyor."

D) siparişler tablosu + CASCADE

SATIR 36-44 göster:

```
FOREIGN KEY (kullanıcı_id) REFERENCES kullanıcılar(id) ON DELETE CASCADE
```

SÖYLE: "Siparişler tablosunda ise ON DELETE CASCADE kullandık. Bu ne demek? Bir kullanıcı silinirse, onun tüm siparişleri de otomatik olarak silinir. SET NULL ve CASCADE arasındaki fark şunu: SET NULL'da ilişki kopar ama kayıt kalır, CASCADE'de bağımlı kayıtlar tamamen silinir."

E) yorumlar — CHECK constraint

SATIR 58-67 göster:

```
puan INT CHECK (puan BETWEEN 1 AND 5),
```

SÖYLE: "Yorumlar tablosunda puan alanında CHECK constraint var. Bu kısıtlama puan değerinin 1 ile 5 arasında olmasını garanti ediyor. 0 veya 6 gibi bir değer girmeye çalışsanız veritabanı bunu reddeder."

12:15-12:45 — Designer'da ER Diyagramı

TIKLA: phpMyAdmin → eticaret_db → ust menu "Designer" tıkla.

SÖYLE: "Burada tablolar arasındaki ilişkilerin görsel halini görüyorsunuz. Okları takip edelim: kullanıcılar tablosundan siparişler tablosuna bir-çok ilişki var, bir kullanıcının birden fazla siparişi olabilir. Siparişlerden sipariş_detaylarına yine bir-çok — her siparişin içinde birden fazla ürün olabilir. Sipariş_detaylarından ürünlere çok-bir ilişki var. Ürünlerden kategorilere çok-bir — her ürün bir kategoriye ait. Yorumlar tablosu hem ürünlere hem kullanıcılara bağlı — bir yorum hem bir ürüne hem bir kullanıcıya ait."

12:45-15:30 — 4 GELİŞMİŞ SQL SORGUSU

DOSYA: VS Code'da queries.sql ac (proje kökü) TIKLA: phpMyAdmin → "SQL" sekmesine tıkla.

SÖYLE: "Şimdi 4 gelişmiş SQL sorgumuzu tek tek göstereceğim. Her birini önce VS Code'da açıklayacağım, sonra phpMyAdmin'de çalıştıracam."

SORGU 1: GROUP BY + JOIN

Amac: Her kategorideki ürün sayısı ve toplam satış

SATIR 28-34 göster:

```
SELECT k.ad AS kategori,
       COUNT(DISTINCT u.id) AS ürün_sayısı,
       COALESCE(SUM(sd.adet * sd.birim_fiyat), 0) AS toplam_satış
FROM kategoriler k
LEFT JOIN ürünler u ON u.kategori_id = k.id
LEFT JOIN sipariş_detayları sd ON sd.ürün_id = u.id
GROUP BY k.id, k.ad
ORDER BY toplam_satış DESC;
```

SÖYLE: "Birinci sorgumuz iki LEFT JOIN ile 3 tabloyu birleştiriyor: kategoriler, ürünler ve sipariş_detayları. LEFT JOIN kullanmamızın sebebi şu: eğer bir kategoride hiç ürün yoksa bile o kategori sonuçlarda görünsün — INNER JOIN kullansaydık boş kategoriler düşecekti. COUNT DISTINCT u.id ile her kategorideki benzersiz ürün sayısını buluyoruz — DISTINCT önemli çünkü aynı ürün birden fazla siparişte geçtiğinde tekrar sayılmasın. SUM ile adet çarpı birim fiyat toplamı satış gelirini veriyor. COALESCE fonksiyonu NULL değerleri sıfıra çeviriyor — hiç satışı olmayan kategoride NULL yerine 0 gözüküyor. GROUP BY ile kategorilere göre grupluyoruz ve ORDER BY ile en çok satış yapan kategori en üstte."

TIKLA: Sorguyu phpMyAdmin SQL sekmesine yapıştır → "Git" tıkla.

SÖYLE: "Sonuç: 6 kategori, en yüksek satışından en düşüğe sıralı."

SORGU 2: SUBQUERY (İÇ İÇE SORGU)

Amac: Ortalama fiyatın üzerindeki ürünler

SATIR 50-54 göster:

```
SELECT id, ad, fiyat,  
       (SELECT ROUND(AVG(fiyat), 2) FROM ürünler) AS genel_ortalama  
FROM ürünler  
WHERE fiyat > (SELECT AVG(fiyat) FROM ürünler)  
ORDER BY fiyat DESC;
```

SÖYLE: "İkinci sorgumuz subquery, yani iç içe sorgu kullanıyor. WHERE koşulundaki parantez içindeki SELECT bir alt sorgudur — önce çalışır ve tüm ürünlerin ortalama fiyatını hesaplar. Sonra dış sorgu bu değerden pahalı olan ürünleri filtreler. SELECT listesine de aynı subquery'yi koyduk ama bu sefer ROUND ile 2 ondalık basamağa yuvarlıyoruz — böylece sonuç tablosunda her satırın yanında genel ortalamayı da görüyorsunuz, karşılaştırma kolaylaşıyor."

TIKLA: Çalıştır.

SORGU 3: GROUP BY + HAVING

Amac: 3'ten fazla siparişi olan müşteriler

SATIR 73-80 göster:

```
SELECT k.id, k.ad, k.email,  
       COUNT(s.id) AS sipariş_sayısı,  
       SUM(s.toplam_tutar) AS toplam_harcama  
FROM kullanıcılar k  
INNER JOIN siparişler s ON s.kullanıcı_id = k.id  
GROUP BY k.id, k.ad, k.email  
HAVING COUNT(s.id) > 3  
ORDER BY toplam_harcama DESC;
```

SÖYLE: "Üçüncü sorgu WHERE ile HAVING arasındaki farkı göstermek için önemli. Burada kullanıcıları siparişleriyle INNER JOIN ile eşleştirip GROUP BY ile grupluyoruz. Kritik nokta şu: neden WHERE değil de HAVING kullandık? WHERE filtreleme gruplamadan ÖNCE yapılır — yani henüz GROUP BY çalışmadan satırları eler. Ama biz 'sipariş sayısı 3'ten büyük' diyoruz, sipariş sayısı COUNT fonksiyonuyla hesaplanıyor, bu bir aggregate fonksiyon. Aggregate fonksiyonlar WHERE içinde kullanılamaz çünkü WHERE çalıştığında henüz gruplama yapılmamış, sayım hesaplanmamıştır. HAVING ise gruplamadan SONRA filtreler, aggregate fonksiyon sonuçlarına erişebilir. Bu yüzden HAVING kullanmak zorundayız."

TIKLA: Çalıştır.

SÖYLE: "3'ten fazla siparişi olan müşteriler, toplam harcamaya göre sıralı."

SORGU 4: KORELASYONLU SUBQUERY

Amac: En çok yorum alan ilk 10 ürün

SATIR 99-104 göster:

```
SELECT u.id, u.ad, u.fiyat,
       (SELECT COUNT(*) FROM yorumlar y WHERE y.ürün_id = u.id) AS yorum_sayısı,
       (SELECT ROUND(AVG(puan), 2) FROM yorumlar y WHERE y.ürün_id = u.id) AS
ortalama_puan
FROM ürünler u
WHERE (SELECT COUNT(*) FROM yorumlar y WHERE y.ürün_id = u.id) > 0
ORDER BY yorum_sayısı DESC, ortalama_puan DESC
LIMIT 10;
```

SÖYLE: "Dördüncü ve en karmaşık sorgumuz korelasyonlu subquery kullanıyor. Normal subquery bir kez çalışır ve tek bir değer döndürür. Ama korelasyonlu subquery dış sorgudaki her satır için ayrı ayrı çalışır. Burada WHERE y.ürün_id = u.id ifadesinde u.id dış sorgudan geliyor — yani her ürün için o ürüne ait yorum sayısı ve ortalama puan ayrı ayrı hesaplanıyor. SELECT listesinde 2 korelasyonlu subquery var: biri yorum sayısını, diğeri ortalama puanı hesaplıyor. WHERE koşulundaki subquery de hic yorumu olmayan ürünleri eliyor. LIMIT 10 ile sadece en popüler 10 ürünü alıyoruz."

TIKLA: Çalıştır.

15:30-17:00 — TRANSACTION (Sipariş Mantiğı)

DOSYA: VS Code → backend/routes/siparişler.js ac

SÖYLE: "Şimdi projenin en kritik backend mantığını göstereceğim: sipariş verme işlemi. Burada MySQL transaction kullanıyoruz. Transaction ne demek? Birden fazla veritabanı işlemini tek bir atomik birim olarak çalıştırmak. Ya hepsi başarılı olur, ya hiçbiri uygulanmaz. E-ticarette bu çok önemli çünkü sipariş verirken birden fazla tabloya yazıyoruz."

A) Transaction başlatma

SATIR 13-14 göster:

```
const conn = await pool.getConnection();
try {
  await conn.beginTransaction();
```

SÖYLE: "İlk adımda connection pool'dan bir bağlantı alıyoruz ve beginTransaction ile transaction'i başlatıyoruz. Bundan sonraki tüm veritabanı işlemleri ya toplu olarak uygulanacak ya da toplu olarak geri alınacak."

B) SELECT FOR UPDATE — satır kilitleme

SATIR 18-22 göster:

```
const [rows] = await conn.query(
  `SELECT id, ad, fiyat, stok FROM ürünler WHERE id IN (${placeholders}) FOR
UPDATE`,
  ids
);
```

SÖYLE: "Bu kısım çok önemli. FOR UPDATE ifadesi ile sipariş edilen ürünlerin satırlarını kilitliyoruz. Bu ne işe yarıyor? Diyelim ki aynı anda iki kişi aynı üründen son 1 adedi almaya çalışıyor. FOR UPDATE olmadan ikisi de stoku 1 olarak görür, ikisi de siparişi verir ve stok eksiye düşer. FOR UPDATE ile ilk gelen satırı kilitler, ikinci gelen birincinin işlemi bitene kadar bekler. Buna race condition koruması deniyor."

C) Stok kontrolü

SATIR 25-30 göster:

```
for (const item of ürünler) {
  const u = ürünMap.get(item.ürün_id);
  if (!u) throw new Error(`Ürün bulunamadi: ${item.ürün_id}`);
  if (u.stok < item.adet) throw new Error(`${u.ad} için yeterli stok yok`);
  toplam += Number(u.fiyat) * item.adet;
}
```

SÖYLE: "Satırlar kilitlendikten sonra stok kontrolü yapıyoruz. Her ürün için istenen adet stoktan fazla mı diye kontrol ediyoruz. Eğer yetersizse throw ile hata fırlatıyor — bu hata catch bloğuna düşer ve rollback tetiklenir. Aynı zamanda toplam tutarı hesaplıyoruz."

D) Sipariş ekleme + Stok düşürme

SATIR 33-47 göster:

```
const [siparişR] = await conn.query(
  'INSERT INTO siparişler (kullanıcı_id, toplam_tutar, durum) VALUES (?, ?, ?)',
  [req.user.id, toplam, 'hazırlanıyor']
);
const siparişId = siparişR.insertId;
for (const item of ürünler) {
  await conn.query(
    'INSERT INTO sipariş_detaylari (...) VALUES (?, ?, ?, ?)',
    [siparişId, item.ürün_id, item.adet, u.fiyat]
  );
  await conn.query('UPDATE ürünler SET stok = stok - ? WHERE id = ?',
    [item.adet, item.ürün_id]);
}
```

SÖYLE: "Üç veritabanı işlemi arka arkaya çalışıyor: önce siparişler tablosuna ana kayıt ekleniyor ve otomatik artan ID alınıyor. Sonra her sipariş edilen ürün için sipariş_detaylari tablosuna bir kayıt ekleniyor. Son olarak her ürünün stoğu UPDATE ile düşürülecek. Bu üç işlem aynı transaction içinde — birinde hata olursa hiçbirisi uygulanmaz."

E) Commit veya Rollback

SATIR 49-55 göster:

```
await conn.commit();
res.json({ sipariş_id: siparişId, toplam_tutar: toplam });
} catch (e) {
  await conn.rollback();
}
```

```
res.status(400).json({ hata: e.message });  
} finally {  
  conn.release();  
}
```

SÖYLE: "Her şey başarılıysa commit çağrılıyor — tüm değişiklikler kalıcı hale geliyor. Herhangi bir adımda hata olursa catch bloğuna düşüyoruz ve rollback çağrılıyor — tüm değişiklikler geri alınıyor, veritabanı transaction başlamadan önceki haline dönüyor. Yarım kalmış sipariş, eksik stok düşüşü gibi tutarsız durumlar kesinlikle olamaz. Finally bloğunda ise connection pool'a bağlantı geri veriliyor."

17:00-17:30 — Raporlar Sayfasında Canlı Demo

TIKLA: Tarayıcı → Admin → sol sidebar'da "Raporlar & SQL" tıkla.

SÖYLE: "Az önce gösterdiğim 4 gelişmiş SQL sorgusu sadece phpMyAdmin'de değil, admin panelimizin Raporlar sayfasında da canlı olarak çalışıyor. Burada her rapor kartında sonuçlar tablo olarak görünüyor ve altında kullanılan SQL sorgusu da açık şekilde gösteriliyor. Toplam 4 zorunlu gelişmiş sorgu ve 1 bonus NOT IN sorgusu mevcut."

TIKLA: Sayfayı aşağı kaydır, tüm raporları göster.

17:30-20:00 — Kapanış + Soru-Cevap

SÖYLE (Egehan): "Projemizi özetleyelim: 6 veritabanı tablosu, 20 sayfa, 18'den fazla React-Bootstrap componenti, JWT ile kimlik doğrulama, bcryptjs ile şifre hashleme, MySQL transaction ile güvenli sipariş yönetimi, 4 gelişmiş SQL sorgusu, Premium üyelik sistemi, öğrenci belge doğrulama akışı, kozmik görsel efektler, çok adımlı checkout akışı, dark ve light mode desteği ve responsive tasarım. Tüm ders gereksinimlerini eksiksiz karşıladık. Sorularınız varsa memnuniyetle cevaplayalım."

Olası Hoca Soruları + Cevaplar

Soru	Cevap	Dosya
Şifreler nasıl korunuyor?	bcryptjs 10 round hash. Plaintext yok. Login'de bcrypt.compare ile doğrulanıyor.	backend/routes/auth.js:13
JWT nasıl çalışıyor?	Login başarılı olunca jsonwebtoken ile 7 gün geçerli token üretiliyor. Her istekte Authorization header'ında gönderiliyor, backend verify ediyor.	backend/middleware/auth.js:5-14
WHERE vs HAVING farkı?	WHERE gruplamadan önce filtreler, ham satırlar üzerinde çalışır. HAVING gruplamadan sonra filtreler, aggregate fonksiyonlara (COUNT, SUM, AVG) erişebilir.	queries.sql:73-80
JOIN vs Subquery farkı?	JOIN tabloları yatay birleştirir, genelde daha performanslı. Subquery iç içe, daha esnek	queries.sql:28, 50

	— özellikle korelasyonlu sorgularda her satır için ayrı çalışabilir.	
Foreign key olmasaydı?	Referans bütünlüğü bozulurdu — varolmayan kategoriye ait ürün kaydedilebilirdi. Bizde ON DELETE CASCADE ve SET NULL ile tutarlılık sağlanıyor.	backend/db/schema.sql:33, 43
Stok yönetimi nasıl?	SELECT FOR UPDATE ile satır kilitleme + Transaction. İki kişinin aynı anda son ürünü almasını engelliyor.	backend/routes/siparişler.js:18-22
SQL injection'a karşı?	Tüm sorgularda prepared statement (? parametreleri) kullanıyoruz. Kullanıcı girdisi doğrudan SQL'e yazılmıyor.	backend/routes/ürünler.js:38-42
Frontend Bootstrap componentleri?	18+ farklı: Navbar, Card, Table, Modal, Form, Button, Alert, Badge, Pagination, Dropdown, Spinner, Carousel, Tabs, Accordion, ListGroup, Container, Row, Col	Tüm frontend/src/pages/
Responsive mi?	Evet. Bootstrap grid sistemi + custom media queries (breakpoint'ler: 600px, 768px, 992px, 1200px).	frontend/src/styles/theme.css
Premium sistem nasıl çalışıyor?	3 plan (Standart/Öğrenci/Aile). Öğrenci planı belge yükleme (JPG/PNG/PDF max 5MB) + üniversite/bölüm/öğrenci no doğrulama gerektiriyor.	frontend/src/pages/Premium.jsx

Acil Durum (sunum sırasında bir şey çökerse)

Sorun	Çözüm
Frontend açılmıyor	Terminal: cd frontend && npm run dev
Backend cevap vermiyor	Terminal: cd backend && npm run dev
MySQL kapalı	brew services start mysql
Port 4000/5173 dolu	lsof -ti :4000
Veriler kayboldu	cd backend && npm run init-db && node db/add-fake-users.js && node db/replace-products.js
Intro çıkmıyor	F12 → konsol: sessionStorage.clear() → yenile
phpMyAdmin yok	XAMPP Control Panel → Apache + MySQL Start

Provayı Yap! Bu rehberi en az 1 kez baştan sona kendiniz çalıştırın. Süre tutun. Geçişler smooth olsun.
Başarılar!